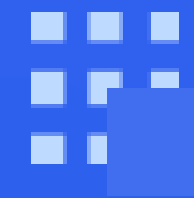




软件测试

Bug间的干涉性



深入分析Bug ■ Bug间的干涉性



- 在软件开发实践中，软件通常存在多个故障（Bug）共存的情况，而非单一故障。
- 多个 Bug 可能使软件的不同功能之间产生冲突，不同 Bug 相互作用还可能引发原本单个 Bug 不会出现的新问题。
- 多个 Bug 同时出现时，产生的错误信息可能会相互混淆，使开发人员难以准确判断问题的根源，这增加了测试和调试的难度。

物理学中的干涉现象

干涉现象的定义

- 在物理学中，干涉是指两个波在同一介质中相遇时发生的现象，分为相长干涉（振幅相加）和相消干涉（振幅相减）。
- 例如，水波相遇时，波峰与波峰相遇形成更高的波峰，波谷与波谷相遇形成更低的波谷。

物理学中“相长干涉”和“相消干涉”

- 如果两个波的振幅具有相同的方向，那么叠加后将形成一个幅度更大的波，称为“相长干涉”。
- 如果两个幅度具有相反的方向，那么叠加后将形成一个幅度较低的波，称为“相消干涉”。



Bug 间干涉的影响

Bug 之间可能会相互干扰，这种现象称为“干涉”，会给程序测试和调试带来风险和障碍。例如，在一个软件中，两个看似无关的Bug可能会相互影响，导致程序出现异常。

Bug 间干涉的定义

对于包含两个故障 f_1 和 f_2 的程序 P ，如果存在测试 t ，使得： t 在 P 上，以及仅包含 f_1 的程序 P_1 和仅包含 f_2 的程序 P_2 上表现各不同，则称 f_1 和 f_2 关于测试 t 相互干涉。

这种定义明确了 Bug 间干涉的条件和表现形式。

Bug 间干涉的类型

- 
- 如果在同一程序中， f_1 和 f_2 同时存在时导致测试 t 失效，而单独存在 f_1 或 f_2 时测试 t 通过，这种现象称为**相长干涉**。例如，两个 Bug 分别导致数据读取错误和数据写入错误，当它们同时存在时，会导致程序完全崩溃。
 - 如果包含故障 f_1 的程序在测试 t 上失效，但当添加另一个故障 f_2 后，测试 t 不再失效，这种现象称为**相消干涉**。例如，一个 Bug 导致程序运行缓慢，另一个 Bug 导致程序提前退出，两者同时存在时，程序反而正常运行。

在实际工程中，可能同时观察到相长干涉和相消干涉两种情况。这增加了软件测试和调试的复杂性，需要综合考虑多种因素。

Bug相长干涉示例

	正确程序 $\mathcal{P}$	有故障 f_1 的程序 $\mathcal{P}$	有故障 f_2 的程序 $\mathcal{P}$	有故障 f_1 和故障 f_2 的程序 $\mathcal{P}$
	<pre>read(a,b); x=a; y=b; if((x+y)==6) print(6); else print(-1);</pre>	<pre>read(a,b); x=a+3; y=b; if((x+y)==6) print(6); else print(-1);</pre>	<pre>read(a,b); x=a; y=b+3; if((x+y)==6) print(6); else print(-1);</pre>	<pre>read(a,b); x=a+3; y=b+3; if((x+y)==6) print(6); else print(-1);</pre>
测试 t_1 输入:a=0 b=0	输出：-1	输出：-1	输出：-1	输出：6
		通过	通过	失败

如上图所示：对于原始程序 $\mathcal{P}$ ，测试 t_1 ， $a=0$ 和 $b=0$ ，预期输出为-1。
注入故障 f_1 的程序输出为-1，注入故障 f_2 的程序输出为-1，因此测试 t_1 均通过。
但是，当 f_1 和 f_2 同时注入后，程序输出为6，测试不通过。
也就是， f_1 和 f_2 这两个故障，关于测试 t_1 ，产生了Bug的相长干涉。

Bug的相消干涉示例

	正确程序 $\mathcal{P}$	有故障 f_1 的程序 $\mathcal{P}$	有故障 f_2 的程序 $\mathcal{P}$	有故障 f_1 和故障 f_2 的程序 $\mathcal{P}$
	<pre>read(a); x=a; y=x-3; print(y);</pre>	<pre>read(a); x=a+1; y=x-3; print(y);</pre>	<pre>read(a); x=a; y=x-4; print(y);</pre>	<pre>read(a); x=a+1; y=x-4; print(y);</pre>
测试 t_1 输入:a=5	输出：2	输出：3	输出：1	输出：2
		失败	失败	通过

如上图所示：对于原始程序 $\mathcal{P}$ ，测试 t_1 ， $a=5$ ，预期输出为2。
注入故障 f_1 的程序输出为3，注入故障 f_2 的程序输出为1，因此测试 t_1 均失效。
但是，当 f_1 和 f_2 同时注入后，程序输出为2，测试通过。
也就是， f_1 和 f_2 这两个故障，关于测试 t_1 ，产生了Bug的相消干涉。

Bug混合干涉示例

如右图所示：
对于原始程序P，
测试t_1， a=0和b=0， 预期输出为2；
测试t_2， a=-4和b=0， 预期输出为2。
讨论单个测试的混合干涉，
和多个测试的混合干涉分析。

	正确程序P	有故障f ₁ 的程序P	有故障f ₂ 的程序P	有故障f ₁ 和故障f ₂ 的程序P
	<pre>read(a,b); x=a; y=b; result= 0 ; if((x+y)==-7) result = 1 ; else result = 2; print(result);</pre>	<pre>read(a,b); x=a-3; y=b; result= 0 ; if((x+y)==-7) result = 1 ; else result = 2; print(result);</pre>	<pre>read(a,b); x=a; y=b-4; result= 0 ; if((x+y)==-7) result = 1 ; else result = 2; print(result);</pre>	<pre>read(a,b); x=a-3; y=b-4; result= 0 ; if((x+y)==-7) result = 1 ; else result = 2; print(result);</pre>
测试t ₁ 输入:a=0 b=0	输出：2	输出：2	输出：2	输出：1
		通过	通过	失败
测试t ₂ 输入:a=-4 b=0	输出：2	输出：1	输出：2	输出：2
		失败	通过	通过

对于测试t₁， 注入故障f₁的程序输出为2， 注入故障f₂的程序输出为2， 均通过。
但是， 当f₁和f₂同时注入后， 程序输出为6， 测试t₁不通过。
也就是， f₁和f₂这两个故障， 关于测试t₁， 产生了Bug的相长干涉。
对于测试t₂， 注入故障f₁的程序输出为1， 测试不通过； 注入故障f₂的程序输出为2， 测试通过。
当f₁和f₂同时注入后， 程序输出为2， 测试通过。
也就是， f₁和f₂这两个故障， 关于测试t₂， 产生了Bug的混合干涉。

深入分析Bug ■ Bug的非单调性

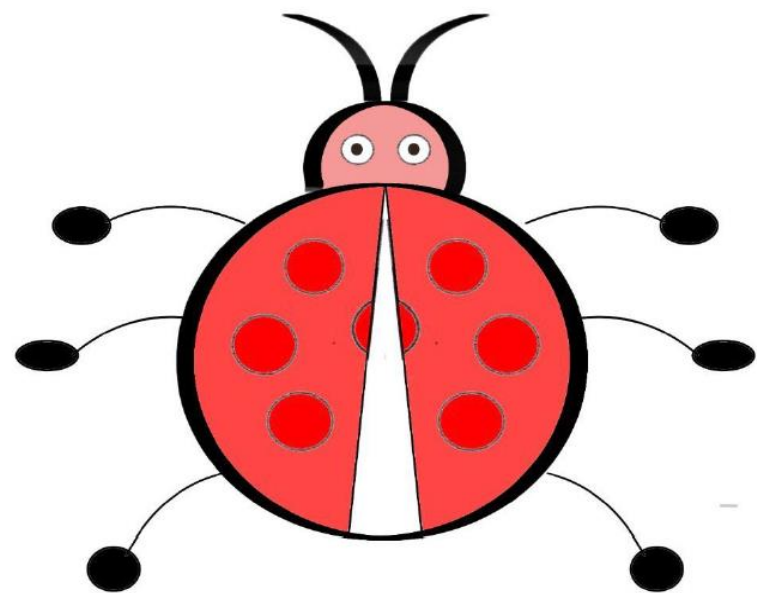
```
int MAX(int x, int y) {  
    int mx = x;  
    if (x < y) {  
        mx = y;  
    }  
    return mx;  
}
```

```
void Mean(double X[], int L) {  
    double mean, sum;  
  
    sum = 0;  
  
    for (int i = 0; i < L; i++) {  
        sum += X[i];  
    }  
  
    mean = sum / L;  
  
    printf("Mean: %f\n", mean);  
}
```

思考与练习:

- 针对MAX程序, 构造Bug的相长干涉, 相消干涉和混合干涉示例, 包括测试集和故障。
- 针对Mean程序, 构造Bug的相长干涉, 相消干涉和混合干涉示例, 包括测试集和故障。
- 前面给出的干涉性是正向分析, 即以一个完全正确程序作为初始点, 尝试构造反向分析示例。

回顾与讨论



Bug间的干涉性

- 相长干涉
- 相消干涉
- 混合干涉

- Bug间的干涉性给程序员的调试和修复带来挑战。
- Bug间的干涉性对测试集的依赖分析。